Project Milestone P2

Analysis and High-Level Design

RoomRadar

Team	Team 02 Section 003
Document version	1.0 - P2 Baseline
Date	11/08/2026
Repository URL	https://github.com/BigKhaz15/RoomRadar
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	Adam Khazem

Team member	NetID	GitHub username
Noor Suhwail	Ncs220001	Nsuhwail20
Raghameena Ramasubbu	RXR230062	MeenaRam23
Tien Nguyen	DAL726120	tnguyen356
Malak Zaarour	DAL028239	Malakzaarour9
Jeimy Rodriguez	JXR200054	JeimyRodriguez
Roshan Palani	DAL872106	roshanpalani2
Adam Khazem	AHK220002	BigKhaz15

0 Revision History, Contribution Record and Changes Since P1

Version	Date	Author	Summary of change
0.3	October 17, 2026	RoomRadar team; Adam Khazem integrated and uploaded	Final P1 baseline covering scope, requirements, initial use cases and project planning.
1.0 — Working Draft	November 5, 2026	RoomRadar team; Adam Khazem integrated and revised	Uploaded the initial P2 working document to `/docs/p2/` for team development.
1.0 — P2 Baseline	November 8, 2026	RoomRadar team; Adam Khazem integrated and revised	Updated the P1 baseline and added use-case analysis, domain and interaction models, high-level architecture, decision records, measured spike evidence and extended traceability.

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Assigned Review Issue
Adam Khazem	**Established the P2 document and repository structure; integrated and revised the document for consistency across requirements, use cases, models, architecture and traceability; committed the replacement reporting spike with AI assistance.**	**Initial P2 document; replacement spike; integrated document in `/docs/p2/`.**	**Malak — Issue #15**
Raghameena Ramasubbu	**Submitted sections 6.1–6.2 and 7.1–7.2 covering quality scenarios, architectural alternatives, architecture decision records and trade-offs.**	**PR #16; commit `ad0a0bd`.**	**Tien — Issue #12**
Tien Nguyen	**Submitted sections 5.2–5.3 containing scenario tables,**	**PR #17; commit `3fa1481`.**	**Noor — Issue #11**

	analysis sequence diagrams and consistency checks.		
Malak Zaarour	**Submitted sections 6.3, 6.5–6.6 and 9.1–9.2 covering components, analysis-to-architecture mappings, design principles, traceability and gap analysis.**	**PR #18; commit `4f5de4c`.**	**Roshan — Issue #14**
Jeimy Rodriguez	**Submitted sections 4.1–4.3 covering selected use cases, expanded main and alternate flows, and self-review findings.**	**PR #19; commit `68e6e08`.**	**Assigned Adam's work; no Adam P2 issue exists.**
Noor Suhwail	**Submitted sections 10–11 covering the project plan, working agreement, milestones, risks and open questions.**	**PR #21; commit `f2c6b06`.**	**Jeimy — Issue #10**
Roshan Palani	**Committed the initial reporting-spike script, README, dependency file and execution log. The integrated document describes the replacement spike committed by Adam.**	**PR #22; commits `7c91a34` (initial spike script) and `4b34645` (execution log).**	**Raghameena — Issue #13**

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Retained the precise acceptance rules in FR-01 and FR-11–15. Expanded their analysis in UC-01/04.	P1 feedback praised expiration, tie-breaking and storage-failure criteria. Their required behavior remains unchanged.	Sections 5–9 model the rules and distinguish verified prototype behavior from outstanding application tests.	Agreed - Nov 7
CH-02	Added the library-service comparison in §1.1. Updated §2.6 with local spike evidence and explicit limits. Retained unresolved student-feedback, room-data and hosting-cost questions in Q1–Q5.	P1 requested evidence supporting need and feasibility and an explanation of value beyond the library's existing service. Section 1.1 now distinguishes reservation availability from student-reported occupancy. The spike provides limited technical evidence; student validation, room-data checks	Feasibility claims distinguish demonstrated prototype behavior from assumptions. Remaining evidence-gathering tasks are tracked in §§10.2 and 11.	Agreed - Nov 8

		and hosting costs remain incomplete.		
CH-03	Added ADR-01–03 and S-01 to investigate the technical approach.	P1 requested a technical check before building on feasibility assumptions.	The SQLite spike supports local acceptance and expiration logic; PostgreSQL, hosting and browser behavior remain unverified.	Agreed - Nov 8
CH-04	Changed FR-07 to SHOULD and FR-08 to COULD. Other functional requirements remain MUST.	P1 feedback requested priorities that protect the schedule. Room details provide a workaround for these filters.	Capacity and amenity filters may be deferred according to priority. Their information remains required in room details under FR-03.	Agreed - Nov 8
CH-05	Expanded UC-01/02/04 in §4; added analysis models in §5 and extended traceability in §9.	P2 requires detailed analysis and visible links from requirements to design. Integration review found inconsistent IDs and missing interactions.	§3.4 retains the diagram and actor-goal list; §3.5 points to §9.1. Tables, diagrams and component mappings now use consistent IDs.	Agreed - Nov 6
CH-06	Expanded the working agreement, responsibilities and risk tracking in §10.	P1 requested meeting cadence, communication channels and response expectations.	Section 10 defines responsibilities, communication and response expectations, escalation procedures, milestones and risk ownership. These rules guide coordination and integration during P3.	Agreed - Nov 7
CH-07	Specified individual branches, commits, PRs and separate integration records in §10.4.	P1 identified limited visibility of individual work in repository snapshots.	Section 0.1 links individual contributions to repository evidence and identifies assigned review issues. Section 10.4 defines individual commits, pull requests and separate integration records. Assigned review issues do not establish completed PR reviews.	Agreed - Nov 5

1 Introduction

1.1 Purpose of this document and of the system

This document establishes the version 1.0 baseline for RoomRadar. It defines the problem, stakeholders, scope, feasibility, requirements and use cases, and extends them with use-case analysis, analysis models, high-level architecture, design decisions, prototype evidence and traceability. It is intended for the course instructor, stakeholders and development team reviewing the proposed system and planning subsequent development.

RoomRadar helps UTD students find suitable study rooms by browsing a team-maintained catalog and filtering by building, capacity, and amenities. Students may anonymously report whether a room is empty or occupied when they observe it; periodic updates are not required. The application displays the latest reported status and its submission time, changing the status to Unknown when the report expires. Occupancy reports provide supplementary information rather than guaranteed availability, and rooms remain searchable even when no current report exists.

UTD's library study-room page provides a calendar for reserving McDermott Library rooms. [R1] RoomRadar's proposed additional value is timestamped, student-reported occupancy information for rooms included in its catalog. Reservation availability and reported physical occupancy represent different information: a booking does not establish whether someone is currently present, and an occupancy report does not grant permission to use a room. RoomRadar does not replace the library's reservation process. Whether students find this additional information useful remains to be validated.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
Browse a team-maintained catalog of UTD study rooms with locations, capacities, and amenities.	Track the individual availability of the seats; the system focuses on the rooms themselves.
Filter rooms by building (MUST), minimum capacity (SHOULD) and amenity (COULD). Capacity and amenity information remain available in room details	Reserve rooms or manage bookings; the system provides information rather than services.

even if their filters are deferred.	
Accept anonymous occupancy reports containing a selected room, an empty or occupied status, and an automatically recorded submission time	Names, usernames, email collection, user accounts, sign-in, and student identity verification; these are not needed for the reporting workflow
Display the latest reported occupancy status and its timestamp	Automatically detect occupancy using sensors or cameras; this would require additional hardware and infrastructure.
Display an Unknown status when no report exists or the latest report has expired	Integrate with already existing UTD booking systems; access to those systems has not been established.
	Administrator interfaces for adding, editing, or removing catalog entries; the development team maintains the catalog outside the application.
	Guarantee current room availability; anonymous reports may be inaccurate or outdated.

Why the exclusions were excluded: These exclusions keep the project achievable within the course timeline by focusing on room discovery and anonymous occupancy reporting using a team-maintained catalog. Reservations, individual seat tracking, automatic occupancy detection, university system integration, user accounts, and administrator interfaces are excluded to limit implementation complexity and external dependencies; current room availability cannot be guaranteed because reports may be inaccurate or outdated.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
User (Student)	A person using RoomRadar to browse and filter study rooms, view reported occupancy, or submit an occupancy report. UTD students are the intended users; student identity is not verified.
UTD	The University of Texas at Dallas, the campus covered by RoomRadar
Occupancy Status	The room’s reported condition, displayed as Reported Empty, Reported Occupied, or Unknown.
Study Room	An enclosed campus space designed for individual or group study.
Room Catalog	The collection of study rooms listed in RoomRadar, including their locations, room identifiers, capacities, and amenities.
Occupancy Report	An anonymous submission containing a selected room, an observation that the room is empty or occupied, and an automatically recorded submission time. No name, username, or email is collected.
Timestamp	The date and time recorded when an occupancy report is submitted
Unknown	A status indicating that no occupancy report exists or the latest report has expired

Expired Report	An occupancy report whose submission timestamp is at least 60 minutes old.
Capacity	The number of people a study room is intended to accommodate.
Amenities	Room features, such as whiteboards or power outlets.

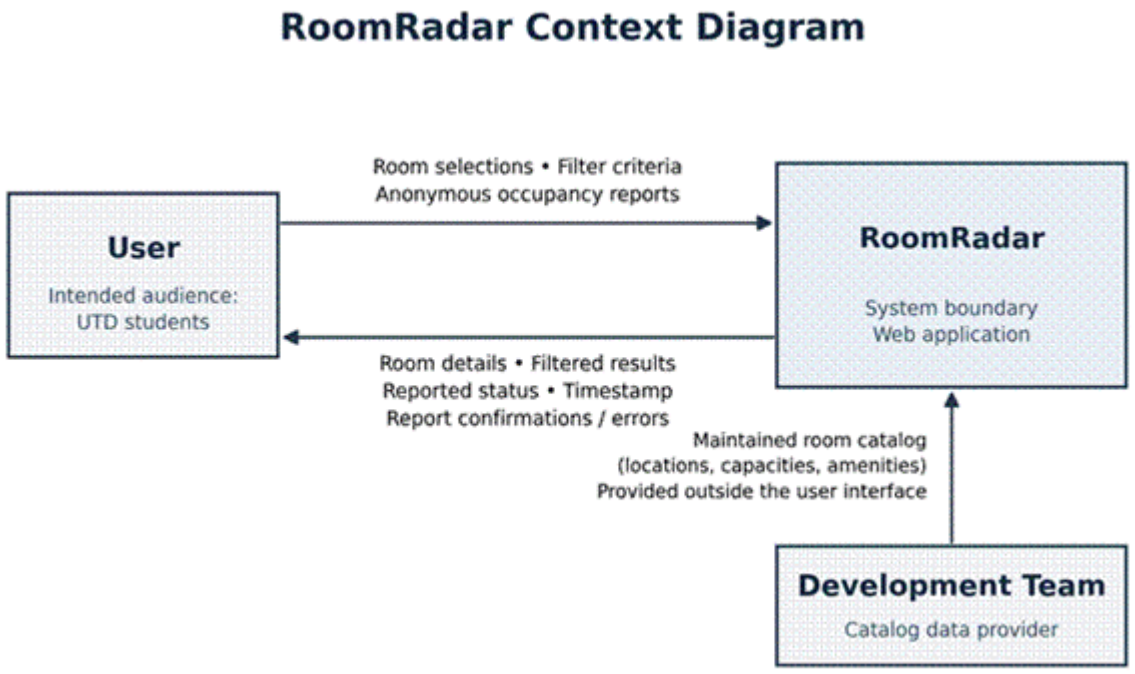
1.4 References

[R1] University of Texas at Dallas, Eugene McDermott Library. Space Availability – Study Rooms. https://libcal.utdallas.edu/reserve/studyrooms.

[R2] University of Texas at Dallas, Eugene McDermott Library. Reserve a Study Room. https://library.utdallas.edu/about-the-library/reserve-a-study-room/

2 Overall Description

2.1 Product perspective and context



RoomRadar is a standalone web application intended for UTD students. Its system boundary includes browsing the room catalog, filtering rooms, accepting anonymous occupancy reports, and displaying reported statuses, timestamps, and Unknown states when reports expire. Users interact with these functions through a web browser without signing in. The development team supplies and maintains the room catalog outside the application's user interface. RoomRadar does not connect to university booking systems, identity services, or occupancy sensors.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
UTD students	Primary intended users who search for study rooms and optionally submit anonymous occupancy reports.	Find rooms matching their location, capacity, and amenity needs, and understand the freshness and limitations of occupancy reports.	Team assumptions based on members' own campus experience; broader student needs have not yet been validated.
Campus facilities and library staff	Non-user stakeholders responsible for spaces listed in the room catalog.	Accurate room descriptions and clear communication that reported occupancy does not grant permission to use or reserve a room.	Team assumptions about staff responsibilities and concerns; no staff consultation has been documented.
Development team	Builds and maintains RoomRadar and supplies the room catalog.	A manageable scope, obtainable room information, and clear, testable requirements.	Team project planning and scope decisions documented in this proposal.

2.3 User characteristics

UTD students are the primary users of RoomRadar. They may have different levels of technical experience and may access the web application on phones while moving between buildings or on laptops while planning a study session. Students need readable room information, straightforward filters, and clearly labeled occupancy reports with timestamps. Status information must be understandable without relying on color alone, and primary controls must support keyboard navigation. Students are not expected to maintain catalog information; the development team supplies and maintains room locations, capacities, and amenities outside the application's user interface.

2.4 Constraints

RoomRadar's first release is subject to the following team-established constraints:

C1 — The application must support browsing and occupancy reporting without user accounts, sign-in, or collection of names, usernames, or email addresses. Source: the anonymous-reporting scope decision in section 1.2.

C2 — The application must operate without integration with university booking systems, university identity services, or physical occupancy sensors. Source: the first-release exclusions in section 1.2.

C3 — Room catalog maintenance must occur outside the application's user interface; the first release will not include an administrator interface. Source: the team-maintained catalog decision in section 1.2.

These are project constraints established by the team, not requirements imposed by UTD.

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold

A1	The team can obtain sufficiently accurate locations, capacities, and amenity information for a limited catalog of UTD study rooms.	Room details and filtering results may be misleading. If suitable information cannot be obtained, the demonstration must use clearly labeled sample data rather than claim to represent actual campus rooms.
A2	Some users will voluntarily submit occupancy reports when they observe rooms; periodic updates are not assumed.	Few rooms will have recent reports, so occupancy status will frequently be Unknown. Catalog browsing and filtering will remain usable.
A3	Voluntary reports will be accurate often enough to provide useful supplementary information. This assumption has not yet been validated.	Incorrect or deliberately false reports may mislead users and reduce trust. Timestamps and expiration limit the age of displayed reports but do not establish their accuracy.
A4	Users have an internet connection when accessing the application or submitting reports.	Users cannot retrieve current shared information or submit reports while disconnected. Offline operation is not planned for the first release.

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Conditionally Feasible	The local spike stored and verified 100/100 reports and passed eight correctness checks covering acceptance, ordering, invalid inputs, rollback and expiration (§8). This supports the core reporting logic. The proposed FastAPI/PostgreSQL deployment and browser behavior remain unverified.
Operational	Not yet validated	The proposed need is based on the team's campus experience; external student feedback has not been documented. Catalog browsing works without reports by design, but the usefulness of occupancy information depends on voluntary participation and accuracy. These uncertainties are tracked in Q3–Q4.
Schedule	Conditionally Feasible	Responsibilities and milestones are defined in §10.1–10.2. Scope excludes bookings, sensors and university-system integration. FR-07 and FR-08 may be deferred according to priority, protecting time for core requirements. Team availability and progress still require monitoring.
Economic / resource	Conditionally feasible	The spike ran locally using Python and SQLite without paid hosting or additional hardware. The planned application requires no occupancy sensors. Hosting and PostgreSQL service costs have not yet been checked, so deployment affordability remains unresolved under Q5.

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-01	The system shall determine each room’s displayed occupancy status from its latest successfully accepted report, using Unknown when no report exists or the report has expired.	MUST	Development team — occupancy-status rules (Section 1.2–1.3)	Using four test rooms, verify that a current empty report produces Reported Empty, a current occupied report produces Reported Occupied, an expired report produces Unknown, and no report produces Unknown.
FR-02	The system shall allow users to submit occupancy reports without creating an account, signing in, or providing identifying information.	MUST	Development team — anonymous-reporting scope decision (Section 1.2)	A user with no account can successfully submit a report by selecting only a room and an Empty or Occupied status.
FR-03	The system shall display the selected room’s building, room identifier, capacity, and amenities as recorded in the catalog.	MUST	UTD students — team-assumed room-information need (Section 2.2)	Select a test room and verify that every displayed field matches its catalog record.
FR-04	The system shall filter the room list by a user-selected building.	MUST	UTD students — team-assumed building-filter need (Section 2.2)	With rooms from Buildings A and B in the catalog, selecting Building A displays all and only rooms in Building A.
FR-05	The system shall display exactly one occupancy label per room: Reported Empty, Reported Occupied, or Unknown.	MUST	Development team — defined occupancy labels (Section 1.3)	Test each of the three states and verify that each room displays exactly one permitted label.
FR-06	The system shall list all catalog rooms when no filters are active, regardless of occupancy status.	MUST	Development team — catalog remains usable without current reports (Section 1.1)	With six catalog rooms covering all three statuses, verify that all six appear when no filters are active.
FR-07	The system shall filter rooms by a user-selected minimum capacity.	SHOULD	UTD students — team-assumed capacity-filter need (Section 2.2)	With room capacities of 2, 4, and 6, selecting a minimum of 4 displays the rooms with capacities of 4 and 6 only.
FR-08	The system shall filter rooms by a user-selected amenity.	COULD	UTD students — team-assumed amenity-filter need (Section 2.2)	Selecting Whiteboard displays all and only rooms whose catalog records include a whiteboard.
FR-09	The system shall display all and only catalog rooms satisfying every active filter when multiple filters are selected.	MUST	Development team — combined-filter behavior	Select Building A, minimum capacity 4, and Whiteboard; verify that the results include all and only rooms meeting all three conditions.

FR-10	The system shall display a no-matching-rooms message when no catalog rooms satisfy the active filters.	MUST	Development team — no-results behavior	Apply a filter combination matching zero rooms and verify that an explicit no-matching-rooms message appears.
FR-11	The system shall store each accepted occupancy report with its selected catalog room, Empty or Occupied status, and an automatically assigned submission timestamp.	MUST	Development team — structured anonymous-reporting scope (Section 1.2)	Submit one Empty report and one Occupied report for different test rooms; verify that each stored report contains the correct room, status, and a system-generated timestamp without requiring the user to enter a time.
FR-12	The system shall treat the most recently accepted report for a room as that room's latest report, resolving equal timestamps by acceptance order.	MUST	Development team — latest-accepted-report precedence rule	Accept an Empty report followed by an Occupied report for the same room and verify that Occupied is selected as the latest report, including when the timestamps are equal.
FR-13	The system shall display the submission date and time of a room's latest accepted report, including after that report expires.	MUST	Development team — report-timestamp display rule	Verify that the displayed date and time match the latest report before and after expiration; a room with no report displays "No reports yet" instead of a timestamp.
FR-14	The system shall change a room's displayed status to Unknown when its latest report reaches 60 minutes of age, including while the room view remains open.	MUST	Development team — proposed 60-minute expiration rule	With no newer report, verify that the reported status remains at 59 minutes 59 seconds and becomes Unknown at 60 minutes without a page reload.
FR-15	The system shall display the outcome of each report-submission attempt as a success confirmation only after storage succeeds or an error message if storage fails.	MUST	Development team — submission success and failure behavior	A successful save produces confirmation; a simulated storage failure produces an error without claiming success or replacing the previously accepted report.

MUST requirements define the minimum release. Minimum-capacity filtering (FR-07) is SHOULD because students can inspect capacity in room details if the filter is deferred. Amenity filtering (FR-08) is COULD because listed amenities provide a similar workaround. FR-09 requires all implemented active filters to work together correctly. The use-case and analysis models describe the intended behavior when these optional filters are included.

3.2 Quality requirements

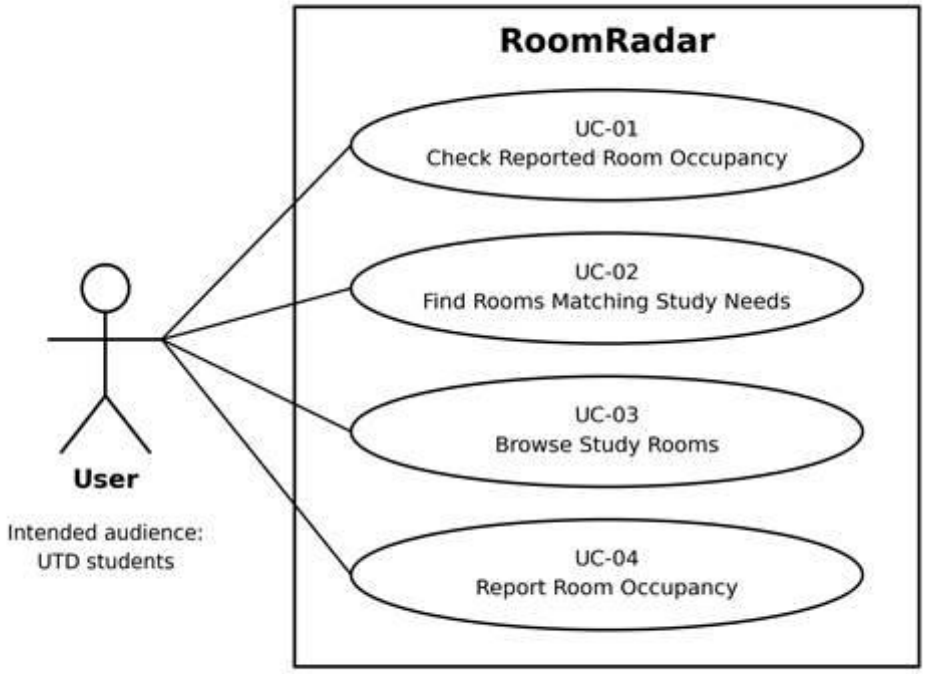
ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	All room-status information shall remain understandable without color, using the text labels defined in FR-05.	MUST	Development team — accessibility decision (Section 2.3)	View all three occupancy states in grayscale and verify that their text labels remain visible and distinguishable.
QR-02	At least 95% of valid report submissions shall display a success confirmation within 3 seconds of submission under a test load of 10 concurrent users.	SHOULD	Development team — proposed reporting-performance target	Run 100 valid submissions with 10 concurrent users in a documented test environment; at least 95 must display confirmation within 3 seconds. Failed submissions count as failures to meet the target.
QR-03	At least 95% of room-list filtering operations shall display the matching results or a no-results message within 5 seconds under a test load of 10 concurrent users and a 100-room catalog.	SHOULD	Development team — proposed filtering-performance target	Run 100 filtering operations using building, capacity, and amenity filters; measure from applying filters to displaying the result. At least 95 must complete within 5 seconds.
QR-04	The occupancy-reporting workflow shall collect and store zero names, usernames, email addresses, passwords, student IDs, or birthdates.	MUST	Development team — anonymous-reporting scope (Section 1.2)	Inspect reporting controls, submitted request data, and stored report records; verify that none of the listed identifying fields are requested, transmitted, or stored by the workflow.
QR-05	The room list, room details, and reporting views shall display without horizontal scrolling at viewport widths of 375, 768, and 1280 CSS pixels.	MUST	Development team — phone and laptop use (Section 2.3)	Inspect each view at all three widths using representative catalog content, including its longest room names and amenity lists; verify zero horizontal overflow.
QR-06	All controls required to browse rooms, apply filters, and submit reports shall be operable using only a keyboard, with a visible focus indicator.	MUST	Development team — keyboard accessibility (Section 2.3)	Complete all three workflows using only the keyboard; verify that every required control is reachable, operable, and visibly focused.

3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system shall support browsing and occupancy reporting without accounts, sign-in, or collection of names, usernames, or email addresses.	Development team — C1, Section 2.4	Anonymous access is an agreed release boundary; changing it requires revising the scope and privacy requirements.

CON-02	The system shall operate without integration with university booking systems, university identity services, or physical occupancy sensors.	Development team — C2, Section 2.4	These integrations are excluded from the agreed release scope and would introduce additional dependencies.
CON-03	The system shall exclude an administrator interface for room catalog maintenance.	Development team — C3, Section 2.4	The team maintains catalog data outside the user interface; adding an administrator interface would expand the agreed scope.

3.4 Initial use cases



ID	Use Case	Summary
UC-01	Check Reported Room Occupancy	The user checks a selected room's reported status and latest report timestamp. Missing or expired reports produce Unknown status.
UC-02	Find Rooms Matching Study Needs	The user filters the catalog by building, minimum capacity, and amenity to find suitable rooms.
UC-03	Browse Study Rooms	The user explores the room catalog and views room locations, capacities, and amenities, regardless of occupancy status.

UC-04	Report Room Occupancy	The user anonymously submits an Empty or Occupied observation for a selected room and receives confirmation or an error.

Expanded use-case descriptions are provided in Section 4.2.

3.5 Requirements to use case traceability

See Section 9.1 for the extended traceability matrix.

4 Use-Case Analysis

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC -02	Find Rooms Matching Study Needs	Combined filters and unknown status	evaluate all rooms
UC -04	Report Room Occupancy	Highest data integrity risk since the students are submitting; errors and orders	Validate rooms/status; report; assign time/order
UC -01	Check Reported Room Occupancy	Checks report freshness and handles missing or expired reports	Find the latest accepted report.

4.2 Expanded use cases

UC- 02 Finding Rooms Matching Study Needs

Primary actor	UTD students
Supporting actors and external systems	None
TUCBW (this use case begins when)	The student chooses to find study rooms using room filters.
TUCEW (this use case ends when)	The student sees matching rooms or a no-matching-rooms message, with room details displayed if the student selects a result.
Preconditions	The room catalog is accessible.
Postconditions	The displayed results satisfy every active filter, or all catalog rooms are displayed when no filters are active. No catalog records or occupancy reports are changed.
Traces to requirements	FR-01, FR-03, FR-04, FR-05, FR-06, FR-07, FR-08, FR-09, FR-10, FR-13; QR-01, QR-03, QR-05, QR-06

Actor input / action	System response

(0) The student views the room catalog.	The system displays all catalog rooms with their reported occupancy labels and the available building, minimum-capacity and amenity filters.
(1) The student selects one or more filters.	The system shows the active filter selections.
(2) The student requests matching rooms.	The system displays all and only catalog rooms satisfying every active filter, regardless of occupancy status. Each result shows its room identifier and exactly one occupancy label: Reported Empty, Reported Occupied or Unknown.
(3) The student selects a matching room.	The system displays the room's building, room identifier, capacity, amenities, reported occupancy label and latest report's submission date and time. The use case ends.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	2	No rooms satisfy every active filter.	The system displays "No matching rooms" and retains the selected filters.	Ends; the student may restart at step 1.
A2	1	The student clears all filters.	The system displays all catalog rooms, regardless of occupancy status.	Rejoins step 3 if the student selects a room; otherwise ends.
A3	3	The selected room has no accepted reports.	The system displays Unknown and "No reports yet" alongside the room details.	Ends.
A4	3	The selected room's latest accepted report is at least 60 minutes old.	The system displays Unknown and retains that report's submission date and time alongside the room details.	Ends.
A5	2	Matching rooms are displayed, but the student does not select one.	The matching results remain displayed.	Ends.

UC- 04 Report Room Occupancy

Primary actor	UTD students
Supporting actors and external systems	None
TUCBW (this use case begins when)	Student chooses to share if the room is empty or occupied
TUCEW (this use case ends when)	The student sees a success confirmation or an error message.
Preconditions	A catalog room is selected, the application is reachable, and the student has observed the selected room.

Postconditions	On success, exactly one report is accepted and a confirmation is displayed. On failure, no report is accepted and a failure message is displayed.
Traces to requirements	FR-01, FR-02, FR-05, FR-11, FR-12, FR-13, FR-15; QR-01, QR-02, QR-04, QR-05, QR-06; CON-01.

Actor input / action	System response
(0) The student views a selected catalog room.	The system displays the room identifier and the Empty and Occupied reporting options.
(1) The student selects Empty or Occupied.	The system displays the selected observation for confirmation.
(2) The student submits the observation.	The system checks that the selected room exists in the catalog and the observation is Empty or Occupied.
(3) — No additional student action.	The system stores the accepted report with the selected room, observation, system-assigned submission timestamp and acceptance order.
(4) — No additional student action.	After storage succeeds, the system displays a success confirmation and the room's status and timestamp derived from its latest accepted report. The use case ends.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	2	The selected room is no longer in the catalog.	The system rejects the submission and displays a message explaining that the room is unavailable for reporting. No report is accepted.	Ends.
A2	2	The observation is missing or is neither Empty nor Occupied.	The system rejects the submission and asks the student to select Empty or Occupied. No report is accepted.	Rejoins step 1.
A3	3	Storage fails.	The system displays an error message without claiming success. No new report is accepted, and the previously accepted report remains unchanged.	Ends.

UC- 01 Checking Reported Room Occupancy

Primary actor	Student
Supporting actors and external systems	None
TUCBW (this use case begins when)	The student requests the reported occupancy of a selected catalog room.

TUCEW (this use case ends when)	The student leaves the selected room’s occupancy view.
Preconditions	A catalog room is selected and its information is accessible.
Postconditions	While the view was open, the displayed label reflected the latest accepted report retrieved by the system and its expiration state. No catalog records or occupancy reports were changed.
Traces to requirements	FR-01, FR-05, FR-12, FR-13, FR-14; QR-01, QR-05, QR-06

Actor input / action	System response
(0) The student views a selected room’s summary.	The system displays the room identifier and an option to view its reported occupancy.
(1) The student requests the room’s reported occupancy.	The system retrieves the latest accepted report, using acceptance order to resolve equal timestamps.
(2) — No additional student action.	If the report is less than 60 minutes old, the system displays Reported Empty or Reported Occupied according to its observation, together with its submission date and time.
(3) The student keeps the occupancy view open.	When the displayed report reaches 60 minutes of age, the system changes the label to Unknown without a page reload and retains the report’s submission date and time.
(4) The student leaves the occupancy view.	The system closes the view. The use case ends.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	1	No accepted report exists for the room.	The system displays Unknown and “No reports yet.”	Rejoins step 4.
A2	2	The latest accepted report is already at least 60 minutes old.	The system displays Unknown and retains the report’s submission date and time.	Rejoins step 4.
A3	3	The student leaves before the report expires.	The system closes the occupancy view; no further update to that closed view is required.	Ends.

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-02	Main flow covers the stated goal and required filtering behavior.	The flow omitted amenity selection and suggested that matching rooms were necessarily available.	Included building, minimum-capacity and amenity filters; specified that results satisfy every active filter regardless of occupancy status.

UC-02	Alternate flows identify their conditions and outcomes.	Clearing filters, expired reports and ending without selecting a room were not explicitly covered.	Added alternate flows for these conditions with clear end or rejoin points.
UC-04	Success acknowledgment follows successful storage.	Acceptance, storage and confirmation were not clearly distinguished. A filter-clearing alternate belonged to a different use case.	Placed confirmation after successful storage, removed the unrelated alternate and specified rejection and storage-failure outcomes.
UC-04	Requirement links cover the behavior described.	Links omitted report storage, acceptance ordering and submission outcomes.	Added FR-11, FR-12 and FR-15 to the requirement links.
UC-01	Beginning, ending and alternate paths are consistent.	The beginning described an outcome, and the expired-report alternate pointed back to the fresh-report display.	Defined the initiating request and exit condition; corrected missing-report and expired-report paths.
UC-01	Expiration is visible while the view remains open.	The flow did not clearly distinguish expiration while viewing from leaving before expiration.	Specified that the status changes to Unknown at 60 minutes while the view remains open, retains the report timestamp, and ends view updates if the student leaves before expiration.

Review performed by Adam Khazem and Jeimy Rodriguez during final P2 integration, November 8, 2026.

5 Analysis Models

5.1 Domain model

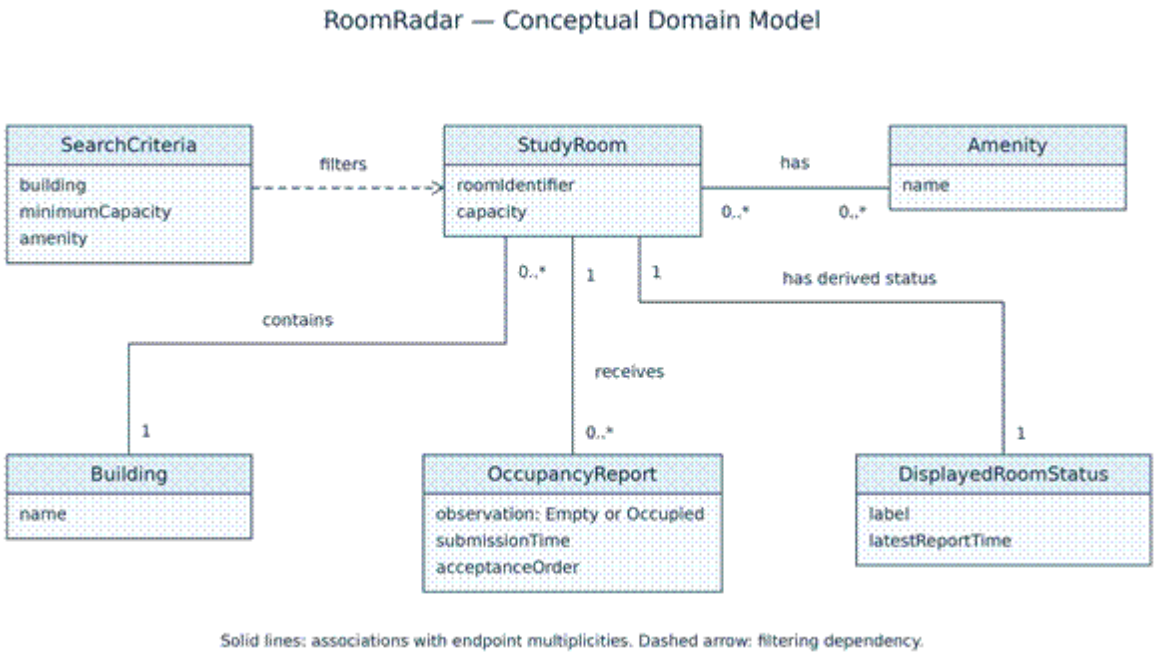


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Building	A campus building	name	One Building contains 0..* StudyRooms; each StudyRoom belongs to 1 Building.
StudyRoom	Cataloged study space	roomIdentifier, capacity	Each StudyRoom has 0..* OccupancyReports; 0..* Amenities; one derived displayed status.
Amenity	A feature available in a study room	name	0..* StudyRooms possess 0..* Amenities.
SearchCriteria	A user's selected filters for finding study rooms	building, minimumCapacity, amenity	Filters StudyRooms using the selected criteria; no persistent association is maintained.
OccupancyReport	A submitted observation of a study room's occupancy	observation (Empty/Occupied), submissionTime, acceptanceOrder	Each report references exactly one StudyRoom.

DisplayedRoomStatus	The current occupancy status shown for a study room	label, latestReportTime	Exactly one current display for each viewed StudyRoom; may be Unknown

5.2 Object interaction models

5.2. UC-02 — Find Rooms Matching Study Needs

(a) Non-trivial steps

- **Combined filtering:** identify all catalog rooms satisfying every active building, minimum-capacity and amenity filter.
- **Occupancy lookup:** retrieve each matching room's latest accepted report, resolving equal timestamps by acceptance order.
- **Status determination:** derive Reported Empty, Reported Occupied or Unknown using the report and its age.
- **Room details:** retrieve the selected room's building, identifier, capacity and amenities.

Showing the initial catalog controls is a presentation step and is not expanded separately below.

Declared interaction object: *RoomCatalog* coordinates this use case. It is an analysis controller, not an additional domain entity.

(b) Scenario table.

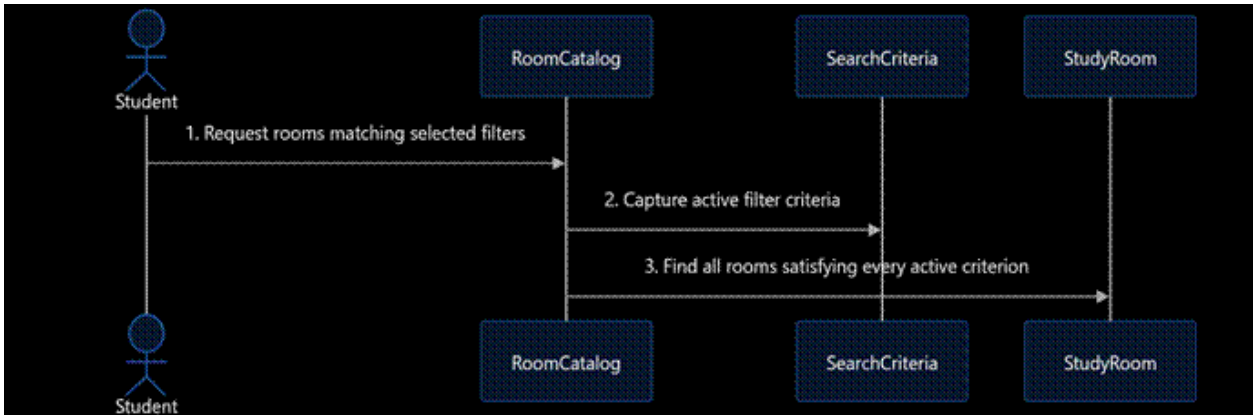
Step	Subject (who acts)	Subject action (what it does)	Other data/objects (what is used)	Object acted upon (who receives)
1	Student	Request rooms matching selected filters	Building, minimum capacity, amenity	RoomCatalog
2	RoomCatalog	Capture the active filter criteria	Selected filter values	SearchCriteria
3	RoomCatalog	Find all rooms satisfying every active criterion	SearchCriteria; Building; Amenity; room capacity	StudyRoom
4	RoomCatalog	Retrieve the latest accepted report for each matching room	Matching room identifiers; submissionTime; acceptanceOrder	OccupancyReport
5	RoomCatalog	Determine each matching room's displayed status	Latest report or absence of a report; current time; 60-minute expiration rule	DisplayedRoomStatus
6	RoomCatalog	Present matching rooms with their reported occupancy labels	Matching StudyRooms; DisplayedRoomStatus	Student
7	Student	Select a matching room	roomIdentifier	RoomCatalog

8	RoomCatalog	Retrieve the selected room's details	roomIdentifier	StudyRoom
9	RoomCatalog	Present the selected room's details, occupancy label and latest report time	Selected room details; DisplayedRoomStatus; latestReportTime or "No reports yet"	Student

Alternate-path coverage: With no active filters, step 3 returns all catalog rooms. With no matching rooms, step 6 displays "No matching rooms," and steps 7–9 do not occur. Missing or expired reports produce Unknown at step 5; an expired report retains its timestamp.

(c) Analysis sequence diagram.

UC-02 — Part A: Apply room filters



UC-02 — Part B: Show results and selected-room details

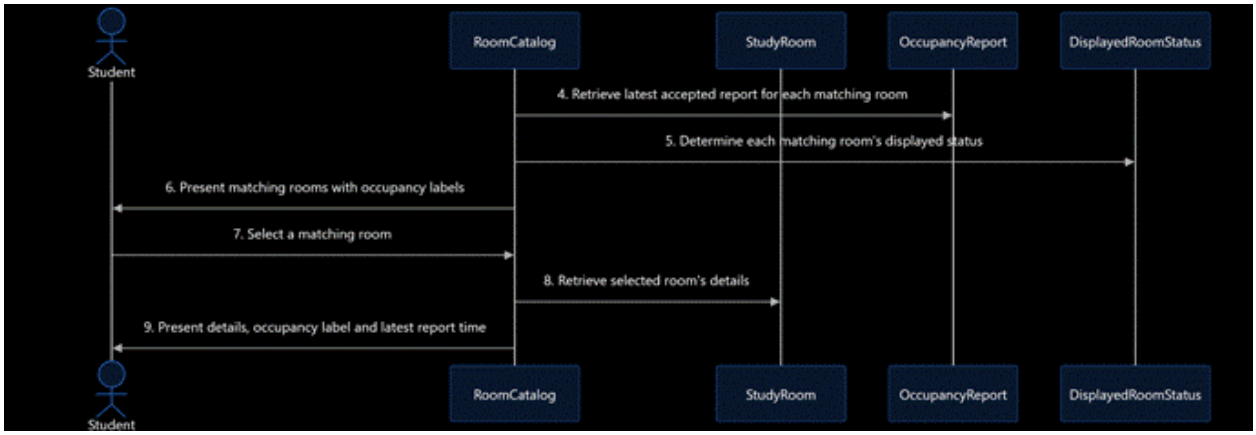


Figure 5.2a. UC-02 analysis interaction, shown in two consecutive panels. Messages 1–9 correspond to scenario-table rows 1–9. The diagram shows the successful path where matching rooms exist and the student selects one.

5.2. UC-01 — Check Reported Room Occupancy

(a) Non-trivial steps

- **Latest-report lookup:** retrieve the selected room's latest accepted report, resolving equal timestamps by acceptance order.
- **Status determination:** derive Reported Empty, Reported Occupied or Unknown from the report's observation and age.
- **Expiration while viewing:** change the displayed label to Unknown when the report reaches 60 minutes of age, retaining its timestamp without requiring a page reload.

Displaying the selected room's identifier before the request is a presentation step and is not expanded separately below.

Declared interaction object: *OccupancyCoordinator* coordinates report lookup, status determination and expiration updates while the view remains open. It is an analysis controller, not an additional domain entity.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data/objects (what is used)	Object acted upon (who receives)
1	Student	Request the selected room's reported occupancy	roomIdentifier	OccupancyCoordinator
2	OccupancyCoordinator	Retrieve the room's latest accepted report	roomIdentifier; submissionTime; acceptanceOrder	OccupancyReport
3	OccupancyCoordinator	Determine the reported occupancy label	Latest accepted report; current time; 60-minute expiration rule	DisplayedRoomStatus
4	OccupancyCoordinator	Present the reported occupancy label and submission date and time	DisplayedRoomStatus; latest report timestamp	Student
5	OccupancyCoordinator	Reevaluate the displayed report when it reaches 60 minutes of age while the view remains open	Displayed report; current time; expiration threshold	DisplayedRoomStatus
6	OccupancyCoordinator	Present Unknown while retaining the report's submission date and time	Updated DisplayedRoomStatus; unchanged report timestamp	Student
7	Student	Leave the occupancy view	Selected room view	OccupancyCoordinator
8	OccupancyCoordinator	End expiration updates for the closed view	View state	OccupancyCoordinator

Alternate-path coverage: If no report exists at step 2, step 4 displays Unknown and "No reports yet," and steps 5–6 are skipped. If the report is already expired at step 3, step 4 displays Unknown with the retained report timestamp, and steps 5–6 are skipped. If the student leaves before expiration, the interaction proceeds directly to steps 7–8.

(c) Analysis sequence diagram.

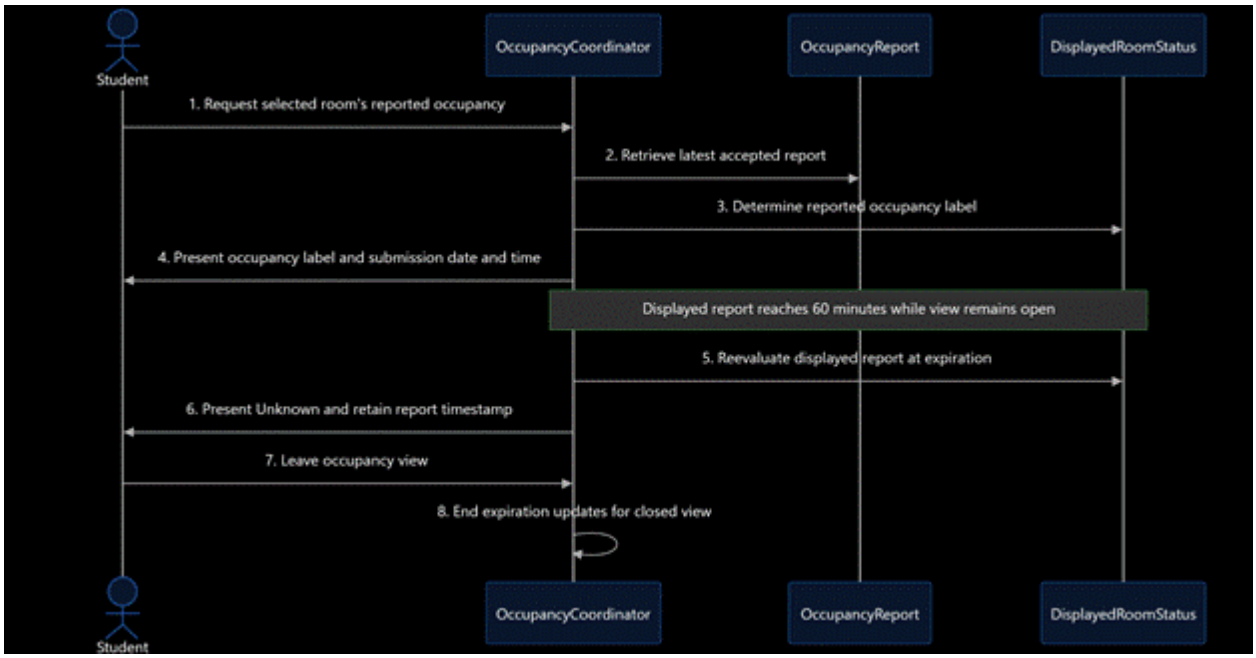


Figure 5.2b. UC-01 analysis sequence diagram for a current report that expires while the occupancy view remains open. Messages 1–8 correspond to scenario-table rows 1–8. Missing reports, already-expired reports and early departure are covered below the scenario table.

5.2. UC-04 — Report Room Occupancy

(a) Non-trivial steps

- **Input validation:** verify that the selected room exists in the catalog and the observation is Empty or Occupied.
- **Report acceptance:** store the report with a system-assigned submission timestamp and acceptance order as one atomic operation.
- **Status determination:** identify the latest accepted report and derive the room's displayed occupancy label.
- **Outcome acknowledgment:** confirm success only after storage succeeds; otherwise display an error.

Displaying the reporting options and the selected observation before submission are presentation steps and are not expanded separately below.

Declared interaction object: *ReportCoordinator* coordinates validation, acceptance and acknowledgment. It is an analysis controller, not an additional domain entity.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	Student	Submit the selected occupancy observation	roomIdentifier; observation	ReportCoordinator
2	ReportCoordinator	Verify that the selected room exists	roomIdentifier	StudyRoom

3	ReportCoordinator	Validate that the observation is Empty or Occupied	observation	ReportCoordinator
4	ReportCoordinator	Store and accept the report atomically with a system-assigned timestamp and acceptance order	Validated roomIdentifier; observation; system time	OccupancyReport
5	OccupancyReport	Return successful acceptance of the stored report	Accepted report	ReportCoordinator
6	ReportCoordinator	Retrieve the room's latest accepted report	roomIdentifier; submissionTime; acceptanceOrder	OccupancyReport
7	ReportCoordinator	Determine the room's displayed occupancy status	Latest accepted report; current time; 60-minute expiration rule	DisplayedRoomStatus
8	ReportCoordinator	Confirm successful submission and present the room's reported status and timestamp	Acceptance outcome; DisplayedRoomStatus; latest report timestamp	Student

Alternate-path coverage: If the room check at step 2 or observation validation at step 3 fails, ReportCoordinator displays the corresponding error and does not attempt storage. If storage fails at step 4, ReportCoordinator displays an error without confirming success or replacing the previously accepted report. Steps 5–8 occur only after successful storage.

(c) Analysis sequence diagram.

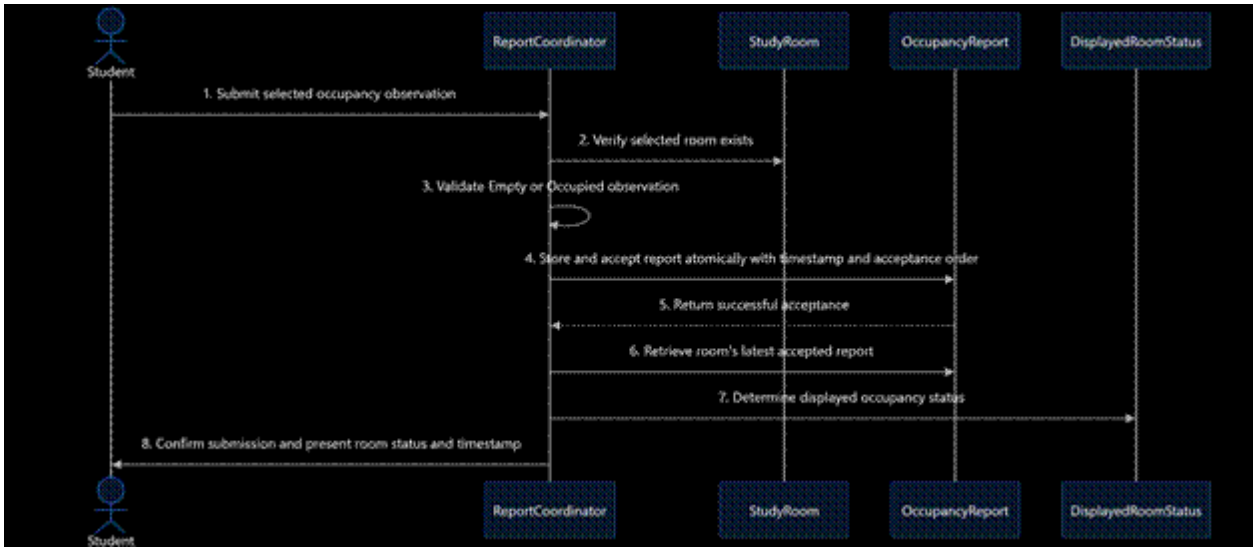


Figure 5.2c. UC-04 analysis sequence diagram for successful occupancy reporting. Messages 1–8 correspond to scenario-table rows 1–8. Validation and storage-failure paths are described below the scenario table.

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 — Every domain data item supplied by the actor in section 4.2 is represented in section 5.1.	Passed — roomIdentifier, observation, building, minimumCapacity and amenity are represented in the domain model.	§5.1, StudyRoom, OccupancyReport and SearchCriteria
K2 — Every sequence-diagram object is a domain class or a declared interaction object.	Passed: declared RoomCatalog, ReportCoordinator and OccupancyCoordinator.	§5.2, controller declarations
K3 — Every numbered sequence-diagram message corresponds to exactly one scenario-table row.	Passed after correcting the scenario tables — UC-02 messages 1–9, UC-01 messages 1–8 and UC-04 messages 1–8 each correspond to one scenario-table row.	§5.2, scenario tables and corresponding sequence diagrams
K4 — Every non-trivial step identified in section 5.2(a) has corresponding scenario-table coverage.	Passed after adding validation, report acceptance and expiration interactions.	§5.2, UC-02 steps 2–5; UC-04 steps 2–7; UC-01 steps 2–6
K5 — Every analyzed use case traces to at least one functional requirement.	Passed after correcting requirement links and use-case IDs.	§4.2 requirement links; §5.2 headings
K6 — The concluding interactions establish the use-case postconditions.	Passed for the documented scenarios: matching results, confirmed storage and expiration updates.	§5.2, UC-02 step 9; UC-04 step 8; UC-01 steps 6–8

Review performed by Adam Khazem and Tien Nguyen during final P2 integration, November 8, 2026.

6 High-Level Design (Architecture)

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	Ten concurrent students submit 100 valid occupancy reports in total during the measurement run.	Reporting service and reporting UI	Stores each accepted report and displays confirmation after storage succeeds.	At least 95 of 100 submissions display success confirmation within 3 seconds of submission. Failed submissions	QR-02

					count as unsuccessful measurements.	
QAS-02	Performance	Ten concurrent students perform 100 filter operations in total against a catalog containing 100 rooms.	Catalog service and filtering UI	Displays rooms satisfying every active filter or a no-matching-rooms message.	At least 95 of 100 operations display results within 5 seconds of the request. Failed operations count as unsuccessful measurements.	QR-03
QAS-03	Privacy	A student submits an anonymous occupancy report.	Reporting inputs, request payloads and stored reports	Accepts only a room identifier and an Empty or Occupied observation as user-supplied report data.	Zero names, usernames, emails, passwords, student IDs or birthdates in reporting input fields, accepted report payloads or stored reports.	QR-04; CON-01
QAS-04	Usability/accessibility	A student browses, filters or reports using a keyboard or a viewport width of 375, 768 or 1280 pixels.	Room browsing, filtering and reporting UI	Keeps the workflows readable and operable at the specified widths and through keyboard navigation.	No horizontal scrolling at the specified widths. All controls required for these workflows are keyboard-operable with a visible focus indicator.	QR-05; QR-06
QAS-05	Correctness/freshness	The displayed report reaches 60 minutes of age while the room view remains open, with no newer report retrieved.	Status Resolver and room occupancy UI	Changes the displayed label to Unknown while retaining the report's submission timestamp.	At 3,600 seconds of report age, the label changes to Unknown without a page reload. The timestamp remains visible, and exactly one permitted occupancy label is displayed.	FR-01; FR-05; FR-13; FR-14

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Layered client-server with a single backend and repository boundary	Centralizes report validation, acceptance ordering and status rules. Separates the UI from application rules and storage. Supports the performance targets without	Backend availability affects all users. The team must manage deployment, database access and write coordination.	Chosen — ADR-01. Appropriate for the bounded scope; performance

	calls between separately deployed backend services.		requires measurement. (ADR-01)
Microservices	Allows catalog and reporting services to be deployed and scaled independently.	Adds service communication, deployment, monitoring and consistency work. Independent scaling is not currently required by the stated workload.	Rejected for this release. Added complexity is not justified by QAS-01 and QAS-02.
Event-driven architecture with server-pushed updates	Can notify open views immediately when another student submits a report.	Requires persistent connections or subscriptions, reconnection handling and additional failure handling.	Rejected for this release. Immediate cross-user updates are outside the current requirements.
Client-only application with locally stored reports	Supports local catalog browsing and filtering with minimal hosting infrastructure.	Reports remain on individual devices; there is no shared authoritative report history or acceptance order.	Rejected. Does not support shared occupancy reporting and FR-12's acceptance-order rule.

RoomRadar uses a layered client-server architecture consisting of a responsive browser UI, a single backend API and a private data store accessed through a repository boundary. The backend validates anonymous submissions and assigns report timestamps and acceptance order. The Status Resolver derives occupancy labels from accepted reports. The browser updates an open view when its displayed report expires, using expiration information supplied by the server. This design supports the project's scope while keeping deployment and communication manageable. Meeting QAS-01 and QAS-02 remains subject to performance measurement; selecting the architecture alone does not demonstrate that those targets have been met.

6.3 Component view

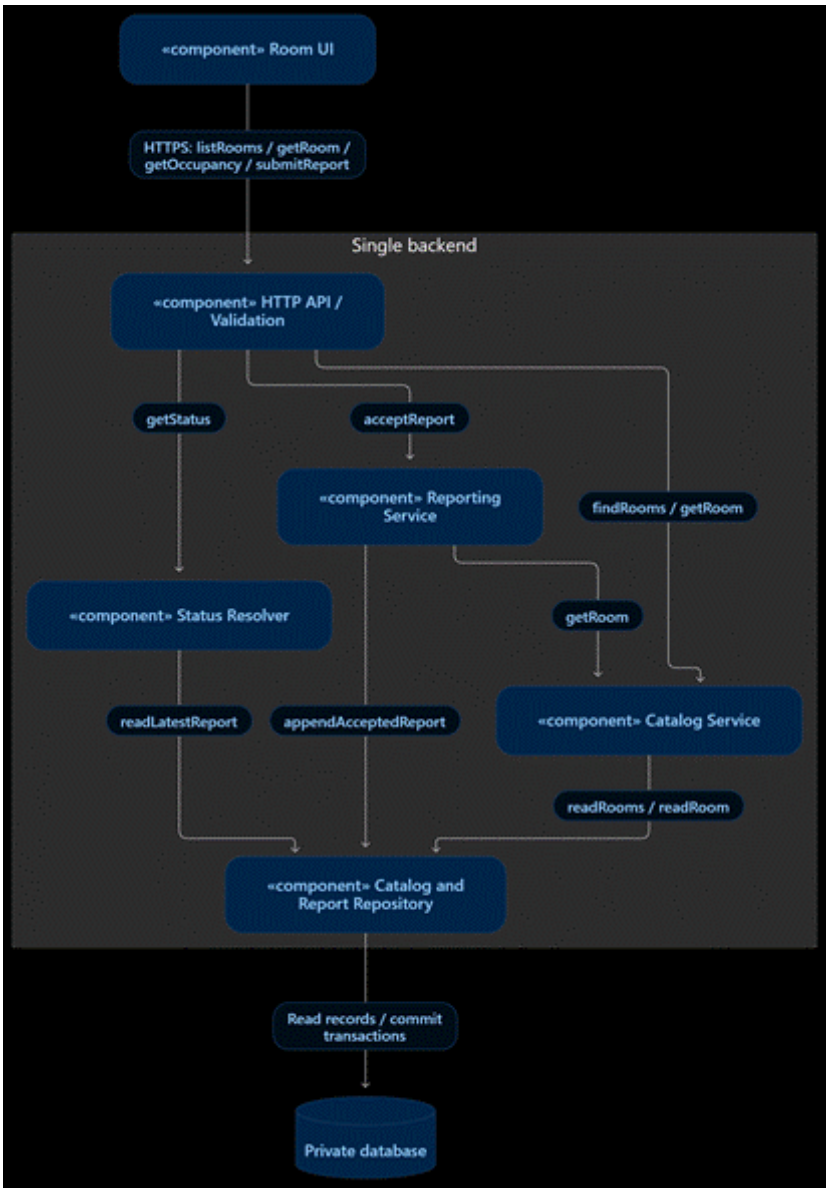


Figure 6.3. RoomRadar component dependencies and required interfaces. The browser accesses the backend through the HTTP API. Backend services access stored data through the repository. The Room UI handles expiration of the displayed report using server-supplied timing information.

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Room UI	Displays rooms, filters, details and submission outcomes; changes an open view's label to Unknown when its displayed report expires.	Student-facing browsing, filtering, occupancy viewing and reporting controls	HTTP API; server-supplied report age or remaining lifetime	UC-01–04; QAS-04/05; ADR-03
HTTP API / Validation	Validates request structure, rejects unsupported reporting fields and returns	listRooms(criteria), getRoom(roomIdentifier), getOccupancy(roomIdentifier), submitReport(roomIdentifier, observation)	Catalog Service; Reporting Service;	UC-01–04; QAS-03; ADR-01

	consistent success or error responses.		Status Resolver	
Catalog Service	Retrieves catalog information and selects rooms satisfying every active filter.	findRooms(criteria), getRoom(roomIdentifier)	Catalog and Report Repository	UC-02/03; QAS-02; ADR-01
Reporting Service	Validates the room and observation, then coordinates atomic report acceptance before acknowledging success.	acceptReport(roomIdentifier, observation)	Catalog Service; Catalog and Report Repository	UC-04; QAS-01/03; ADR-02
Status Resolver	Derives the displayed label, latest report timestamp and remaining lifetime from the latest accepted report and authoritative server time.	getStatus(roomIdentifier)	Catalog and Report Repository; server clock	UC-01/02/04; QAS-05; ADR-03
Catalog and Report Repository	Reads catalog records and atomically stores accepted reports with server-assigned timestamps and acceptance order.	readRooms(), readRoom(roomIdentifier), appendAcceptedReport(roomIdentifier, observation), readLatestReport(roomIdentifier)	Private database; server clock	UC-01–04; ADR-01/02

The repository returns successful report acceptance only after the storage transaction commits. The Reporting Service does not acknowledge a failed write as successful. Database credentials remain on the backend. The Room UI uses the server-supplied remaining report lifetime to schedule the change to Unknown without a page reload, retaining the report's submission timestamp.

6.4 Deployment view

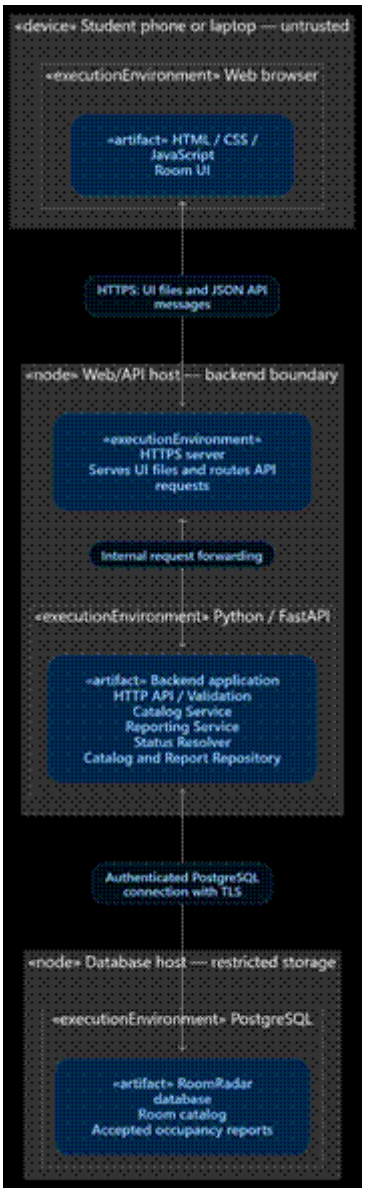


Figure 6.4. Proposed RoomRadar deployment. *The Room UI executes in the student's browser. Backend components execute within one Python/FastAPI application. PostgreSQL stores catalog records and accepted reports behind a restricted database boundary. Browsers cannot connect directly to the database. Hosting providers have not yet been selected.*

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
Student's phone or laptop	Web browser executing the HTML, CSS and JavaScript Room UI	Web/API host through HTTPS; API requests and responses use JSON	Untrusted client device
Web/API host	HTTPS endpoint serving the UI files; Python/FastAPI application containing HTTP API / Validation, Catalog Service, Reporting Service, Status Resolver and Catalog and Report Repository	Browser through HTTPS; database host through an authenticated PostgreSQL connection with TLS	Backend application boundary

Database host	PostgreSQL database containing the room catalog and accepted occupancy reports	Accepts database connections from the backend only; no browser access	Restricted data-storage boundary

Database credentials remain on the backend and are excluded from source control. The database is not directly accessible to students. The team is responsible for application validation, access configuration, dependency maintenance and backup configuration. A hosting provider is responsible for its underlying infrastructure; responsibility for operating-system and database maintenance depends on the selected hosting service. Hosting selection and cost verification remain open until documented evidence is available.

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
Building	Catalog Service	Owns catalog interpretation; the repository handles persistence and retrieval.
StudyRoom	Catalog Service	Owns room details and room-existence checks; the repository stores the records.
Amenity	Catalog Service	Owns amenity information and its use in filtering; the repository stores room–amenity associations.
SearchCriteria	Catalog Service	Owns filter interpretation and matching rules. Room UI captures selections; HTTP API / Validation checks request structure. Criteria are not persisted.
OccupancyReport	Reporting Service	Owns report validation and acceptance. The repository atomically persists each accepted report with its server-assigned timestamp and acceptance order.
DisplayedRoomStatus	Status Resolver	Owns status derivation. Room UI displays the result and handles expiration using server-supplied timing information. Status is derived, not stored as an independent occupancy record.
RoomCatalog	Catalog Service	The analysis controller’s filtering and room-detail responsibilities map here. Status lookup delegates to Status Resolver; presentation belongs to Room UI.

ReportCoordinator	Reporting Service	The analysis controller’s acceptance workflow maps here. Room checks delegate to Catalog Service, storage to the repository, and status derivation to Status Resolver. Room UI displays the outcome.
OccupancyCoordinator	Status Resolver and Room UI	Deliberately split: Status Resolver retrieves and evaluates the latest report; Room UI schedules expiration of the displayed report and stops updates when the view closes.

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Room UI handles presentation, HTTP API / Validation checks request structure, services apply application rules, and the repository handles storage.	§6.3 component view; ADR-01
Information hiding	Database credentials and transaction details remain behind the backend repository boundary. The UI receives report outcomes and status information without accessing the database directly.	§6.4 deployment view; Catalog and Report Repository; ADR-02
Low coupling	Components depend on defined interfaces. Catalog, Reporting and Status services access stored records through the repository rather than issuing database operations independently.	§6.3 provided and required interfaces
High cohesion	Catalog Service handles room discovery, Reporting Service handles report acceptance, and Status Resolver handles occupancy-label derivation.	§6.3 component responsibilities; §6.5 object mappings
Design for change	The expiration policy is evaluated by Status Resolver, which supplies timing information to Room UI. Database access is isolated behind the repository interface.	Status Resolver; Catalog and Report Repository; ADR-03
Keep it simple (KISS)	One backend application serves the bounded project scope. Open views use expiration timers without requiring persistent server-push connections.	§6.2 architectural alternatives; ADR-01/03

7 Design Rationale

7.1 Architecture decision records

ADR-01 Layered client-server with a single backend

Status	Accepted as the P2 design baseline; performance validation remains outstanding.
Date and owner	November 8, 2026 — Raghameena Ramasubbu, architecture section owner; revised by Adam Khazem and Raghameena Ramasubbu during final integration.
Context (drivers)	RoomRadar needs shared anonymous reports, consistent acceptance ordering, combined room filtering and report expiration. QAS-01 and QAS-02 specify a workload of ten concurrent users. The project's limited scope and course timeline favor a small number of deployable parts.
Options considered	Layered client-server with one backend; microservices; event-driven architecture with server-pushed updates; client-only application with locally stored reports.
Decision	Use an HTML/CSS/JavaScript browser UI, one Python/FastAPI backend and a PostgreSQL database. Separate backend responsibilities into HTTP API / Validation, Catalog Service, Reporting Service, Status Resolver and Catalog and Report Repository.
Consequences	Centralizes validation and report rules while keeping deployment manageable. The backend is a shared point of failure, and the team must configure database access, maintain dependencies and coordinate writes. Independent service scaling and immediate cross-user updates are not provided by this design.
Evidence	Section 6.2 compares the alternatives against the project's scope and quality targets. Sections 6.3–6.5 describe component responsibilities, deployment and analysis-object mappings. These provide design rationale; they do not establish that performance targets have been met.
Related	UC-01–04; QAS-01–05; CON-01–03; sections 6.2–6.5.

ADR-02 Server-authoritative acceptance and timestamps

Status	Accepted as the P2 design baseline; locally checked using SQLite. PostgreSQL transaction and concurrency verification remain outstanding.
Date and owner	November 8, 2026 — Raghameena Ramasubbu, architecture section owner; revised by Adam Khazem and Raghameena Ramasubbu during final integration.
Context (drivers)	Multiple students may report the same room. FR-11 requires system-assigned submission timestamps, FR-12 requires the most recently accepted report to take precedence, and FR-15 prohibits success confirmation when storage fails. Reports must remain anonymous.
Options considered	Server-assigned timestamps and acceptance order stored atomically; client-assigned timestamps with last-write-wins behavior; overwriting one mutable occupancy record per room.

Decision	Accept reports through Reporting Service and the repository. Within a database transaction, assign a server timestamp and acceptance order and persist the report. Serialize acceptance for each room so its ordering agrees with successful transaction completion. Acknowledge success only after commit. Select the latest report by acceptance order, including when timestamps are equal.
Consequences	Produces a consistent report order without trusting student device clocks. Requires transaction coordination and error handling. Keeping accepted report history increases storage use. A database-generated sequence number alone is insufficient if concurrent transactions can commit in a different order.
Evidence	S-01 stored and verified 100/100 anonymous reports using ten concurrent request workers. It also verified unique acceptance ordering, precedence for equal timestamps and rollback after an injected write failure. These results support the acceptance mechanism in the local SQLite prototype. PostgreSQL transaction coordination and browser success acknowledgment remain unverified. See section 8 and /spikes/.
Related	FR-02; FR-11; FR-12; FR-15; UC-04; QAS-01/03; CON-01; sections 5.2, 6.3 and 8.

ADR-03 Derive occupancy status and schedule expiration using server-supplied timing

Status	Accepted as the P2 design baseline; expiration logic checked locally. Browser-level expiration verification remains outstanding.
Date and owner	November 8, 2026 — Raghameena Ramasubbu, architecture section owner; revised by Adam Khazem and Raghameena Ramasubbu during final integration.
Context (drivers)	FR-01 requires Unknown when no report exists or the latest report has expired. FR-13 preserves the latest report timestamp after expiration. FR-14 requires an open view to change to Unknown when its displayed report reaches 60 minutes of age. Immediate notification of reports submitted by other students is not required.
Options considered	Derive status when requested and schedule expiration in the browser; store a mutable occupancy label and update it through a scheduled background process; maintain persistent server-push connections.
Decision	Status Resolver derives the label from the latest accepted report and authoritative server time. Its response includes the latest report timestamp and remaining lifetime. Room UI uses that timing information to schedule the transition to Unknown without a page reload, retaining the timestamp. Timing accounts for elapsed request time rather than trusting the device’s wall clock. The UI rechecks status when a suspended view becomes active again.
Consequences	Avoids maintaining a separate stored occupancy label or persistent connections. Requires careful handling of network delay and browser timer suspension. Other students’ reports are reflected when fresh status is requested, rather than pushed instantly to every open view.
Evidence	S-01 verified that a report remains Reported Occupied at 3,599 seconds and becomes Unknown at 3,600 seconds while retaining its timestamp. It also verified Unknown when no report exists. These checks used supplied test times; they did not test live browser timers, network delay or browser suspension. See section 8 and /spikes/.
Related	FR-01; FR-05; FR-13; FR-14; UC-01; QAS-05; sections 5.2, 6.3 and 6.5.

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Implementation simplicity and consistent rules	Use one layered backend with defined service and repository responsibilities.	Backend downtime affects all users; the team must maintain the application and database configuration.	ADR-01
Integrity of accepted reports	Assign timestamps and acceptance order on the server; coordinate acceptance per room and acknowledge only committed reports.	Transaction coordination can delay simultaneous submissions to the same room. Retained report history increases storage use.	ADR -02
Freshness with limited infrastructure	Derive status from accepted reports and schedule open-view expiration using server-supplied timing.	Requires handling network delay and suspended browser timers. New reports are not instantly pushed to other users' open views.	ADR -03

8 Prototype Evidence: Architecture Spike

Roshan Palani and Adam Khazem collaborated on the spike with AI assistance.

Spike ID and title	S-01 — Anonymous Report Acceptance and Expiration
Risk or decision it addresses	Atomic report acceptance and ordering (ADR-02); expiration logic (ADR-03).
Question (answerable yes or no)	Can a local prototype persist concurrent anonymous reports, preserve acceptance order, roll back failed writes and apply the 60-minute expiration rule?
Set-up	Python HTTP server, temporary SQLite database and one sample room; 100 submissions through ten concurrent request workers.
How it was measured	Timed HTTP acknowledgments, checked responses against stored records and tested invalid inputs, failed writes, equal timestamps and expiration boundaries.
Result (numbers)	100/100 reports stored and verified; 100 unique acceptance-order values. Mean HTTP latency: 26.150 ms; p95: 132.227 ms. All eight correctness checks passed.
What it changed or confirmed	Supports atomic acceptance and expiration logic in the local prototype. Does not verify PostgreSQL, browser timers or QAS-01's visible-confirmation target.
Where to see it	/spikes/ — code, README, detailed results and execution log.

9 Traceability

9.1 Extended traceability matrix

“Design” means documented behavior, not a completed implementation test. “S-01” refers to the local SQLite spike in section 8.

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	UC-01/02/04	OccupancyReport; DisplayedRoomStatus	Status Resolver; Room UI	QAS-05; ADR-03	§5.2 status interactions; S-01 checks missing and expired reports.
FR-02	UC-04	ReportCoordinator; OccupancyReport	HTTP API / Validation; Reporting Service	QAS-03; ADR-02	§4.2 anonymous reporting flow; S-01 accepts reports containing only room identifier and observation.
FR-03	UC-02/03	StudyRoom; Building; Amenity; RoomCatalog	Catalog Service; Room UI	ADR-01	§5.2 UC-02 steps 8–9; design only.
FR-04	UC-02	SearchCriteria; Building; RoomCatalog	Catalog Service	QAS-02; ADR-01	§5.2 UC-02 steps 1–3; design only.
FR-05	UC-01/02/04	DisplayedRoomStatus	Status Resolver; Room UI	QAS-05; ADR-03	§5.2 status presentation; S-01 checks Reported Occupied and Unknown. Full label/UI verification remains open.
FR-06	UC-02/03	SearchCriteria; StudyRoom; RoomCatalog	Catalog Service; Room UI	ADR-01	§4.2 UC-02 alternate A2; §5.2 alternate-path coverage. Design only.
FR-07	UC-02	SearchCriteria; StudyRoom; RoomCatalog	Catalog Service	QAS-02; ADR-01	§5.2 UC-02 steps 1–3; design only.

FR-08	UC-02	SearchCriteria; Amenity; RoomCatalog	Catalog Service	QAS-02; ADR-01	§5.2 UC-02 steps 1–3; design only.
FR-09	UC-02	SearchCriteria; StudyRoom; RoomCatalog	Catalog Service	QAS-02; ADR-01	§5.2 UC-02 step 3 applies every active criterion; design only.
FR-10	UC-02	SearchCriteria; RoomCatalog	Catalog Service; Room UI	QAS-02	§4.2 UC-02 alternate A1; §5.2 no-results coverage. Design only.
FR-11	UC-04	OccupancyReport; ReportCoordinator	Reporting Service; Catalog and Report Repository	ADR-02	S-01 verifies 100 accepted responses against persisted room, observation, timestamp and order values.
FR-12	UC-01/02/04	OccupancyReport; ReportCoordinator; OccupancyCoordinator ; RoomCatalog	Catalog and Report Repository; Status Resolver	ADR-02	S-01 checks unique ordering and equal-timestamp precedence. PostgreSQL verification remains open.
FR-13	UC-01/02/04	OccupancyReport; DisplayedRoomStatus	Status Resolver; Room UI	QAS-05; ADR-03	S-01 checks timestamp retention after expiration; displayed date/time formatting remains untested.
FR-14	UC-01	DisplayedRoomStatus; OccupancyCoordinator	Status Resolver; Room UI	QAS-05; ADR-03	S-01 checks status at 3,599 and 3,600 seconds. Live browser expiration remains untested.
FR-15	UC-04	OccupancyReport; ReportCoordinator	Reporting Service; Catalog and Report Repository; Room UI	QAS-01; ADR-02	S-01 checks rollback after an injected write failure; §4.2 defines the error message. Browser acknowledgment remains untested.

QR-01	UC-01/02/04	DisplayedRoomStatus	Room UI	ADR-01	§4.2 specifies textual occupancy labels; UI verification remains open.
QR-02	UC-04	ReportCoordinator; OccupancyReport	Reporting Service; Room UI	QAS-01	S-01 records local HTTP latency only; visible-confirmation timing remains unverified.
QR-03	UC-02	SearchCriteria; RoomCatalog	Catalog Service; Room UI	QAS-02	§6.1 defines the workload and response target; no filtering benchmark completed.
QR-04	UC-04	OccupancyReport; ReportCoordinator	HTTP API / Validation; Reporting Service	QAS-03	S-01 rejects a payload containing student_id. Full application privacy verification remains open.
QR-05	UC-01/02/03/04	SearchCriteria; StudyRoom; DisplayedRoomStatus	Room UI	QAS-04	§6.1 specifies viewport checks; responsive UI testing remains open.
QR-06	UC-01/02/03/04	SearchCriteria; StudyRoom; OccupancyReport	Room UI	QAS-04	§6.1 specifies keyboard operation and visible focus; accessibility testing remains open.
CON-01	UC-01/02/03/04	OccupancyReport; ReportCoordinator	Room UI; HTTP API / Validation	QAS-03; ADR-01/02	§4.2 and §6.3 contain no account or sign-in workflow; S-01 requires no identity fields.
CON-02	UC-01/02/03/04	StudyRoom; OccupancyReport	Catalog Service; Reporting Service	ADR-01	§6.3–6.4 require no university-system or sensor connection; design evidence.

CON-03	UC-02/03	Building; StudyRoom; Amenity	Catalog Service; Catalog and Report Repository	ADR-01	§6.3 exposes catalog reads without a student-facing catalog-administration interface; design evidence.

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	None identified in the revised matrix.	Section 9.1 maps every FR, QR and CON to responsible components. A component assignment does not establish implementation or test completion.
Components with no requirement, QAS or ADR	None identified in the revised component view.	Section 6.3 links each component to use cases, quality scenarios or architecture decisions. Section 6.5 maps the analysis objects to those components.
Use cases not analyzed	UC-03 — Browse Study Rooms	Retained at actor-goal level. Its basic catalog-display behavior is covered by UC-02's initial view and clear-filters alternate. Detailed analysis prioritized filtering, report acceptance and occupancy expiration.
Quality targets not verified	QAS-01, QAS-02 and QAS-04; application-level portions of QAS-03 and QAS-05	Recorded as verification work for P3. The spike does not establish visible-confirmation timing, filtering performance, responsive layout, keyboard accessibility or complete application privacy.
Prototype-to-deployment gap	SQLite prototype versus proposed FastAPI/PostgreSQL deployment	Section 8 explicitly limits the results to the local prototype. PostgreSQL transaction ordering, deployed performance and browser expiration behavior require separate verification.

10 Project Plan

10.1 Roles and working agreement

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
Adam Khazem	Team coordinator and repository lead: coordinate deadlines, maintain the repository, organize contribution evidence, and review document consistency.	Discuss options and evidence; if there's no consensus, vote and record the decision.
Raghameena Ramasubbu	Quality requirements and documentation lead: maintain measurable quality requirements, references, and tooling documentation.	Same team decision process.
Jeimy Rodriguez	Stakeholder and use-case lead: document stakeholder needs, maintain use-case flows, and coordinate feasibility input from the team.	Same team decision process.
Tien Nguyen	Functional requirements and constraints lead: maintain functional requirements, assumptions, dependencies, and release constraints.	Same team decision process.
Roshan Palani	Technical feasibility lead: investigate room data, occupancy-report storage, and the technical approach; document findings and limitations.	Same team decision process.
Noor Suhwail	Project planning lead: maintain milestones, team responsibilities, and the working agreement; flag scheduling issues.	Same team decision process.
Malak Zaarour	Traceability and risk lead: maintain requirement-to-use-case links, support use-case revisions, and track risks and unresolved questions.	Same team decision process.

Topic	Proposed Concrete Rule	Proof / escalation
Communication	Use Discord for coordination and GitHub issues/PRs to record task decisions, blockers and changes affecting other sections.	Record decisions with affected IDs and named owners in the relevant issue.
Meeting cadence	Hold 15-minute check-ins on Monday and Thursday, plus a final review before each submission.	Adam records decisions, owners and due dates within 24 hours. Absent members respond asynchronously.
Response expectations	Acknowledge ordinary mentions within 24 hours on weekdays. Agree on availability before requiring faster responses near deadlines.	If a response is overdue, Adam contacts the owner and adjusts or reassigns dependent work.
Task ownership	Each task has a named owner, deadline and completion condition. Shared tasks identify each member's contribution.	Track assignments and completion in GitHub issues; record actual contributions in section 0.1.

Document editing	Edit the shared Google Doc. Changes identify affected FR/QR/CON/UC/ADR IDs, the reason and impacts on related sections. Keep existing IDs stable.	Record baseline changes in section 0.2 and update affected models and traceability.
Repository contributions	Each member commits their own authored sections or source files from their own GitHub account and opens a PR linked to the assigned issue.	Section 0.1 links actual commits and PRs. Adam's final integration commits are recorded separately.
Review and integration	Adam performs final integration review and checks consistency across sections. Additional reviewers record only reviews they actually perform.	Record findings and corrections in PR comments or review logs. Preserve authorship when merging.
Disagreements	Discuss alternatives using requirements and evidence. If consensus is not reached, use a majority vote, with Adam breaking a tie.	Record the decision, rationale and affected IDs in the relevant issue; update section 0.2 when needed.
Missed tasks	Report blockers or expected delays promptly. Adam contacts the owner and reassigns unfinished work when necessary to protect the deadline.	Record the revised owner and deadline in the issue; credit the person who completes the work.
Completion standard	A task is complete when its stated conditions are met, related sections agree and supporting files are committed. Final submission requires the same PDF in /docs/p2 and Canvas, with the p2 tag on its repository commit.	Check links, repository access, final PDF and tag before submission. Incomplete verification remains explicitly identified.

10.2 Plan to P3

D denotes the official P3 submission deadline published by the instructor. Milestone dates are expressed in calendar days before D.

Milestone	Date	Owner	Done when
Close evidence gaps	D − 10 days	Jeimy Rodriguez; Roshan Palani	Actual student feedback, sample room-data checks and hosting-cost findings are documented with sources and dates. Remaining assumptions are explicitly identified.

Resolve design questions	D − 9 days	Raghameena Ramasubbu; Noor Suhwail	Open questions have decisions or justified deferrals; affected ADRs, requirements and risks are updated.
Complete detailed design	D − 7 days	Tien Nguyen; Malak Zaarour	P3-required design class and sequence diagrams agree with the accepted use cases and architecture.
Verify the selected technical approach	D − 5 days	Roshan Palani	A reproducible FastAPI/PostgreSQL check records report persistence, acceptance ordering and failure behavior, with code and actual results committed.
Update traceability and verification plans	D − 4 days	Malak Zaarour; Jeimy Rodriguez	Requirements link to the relevant detailed-design elements and planned verification; gaps are documented.
Complete cross-section review	D − 2 days	Adam Khazem; section owners	Review findings are recorded, contradictions are resolved and corrected contributions are merged.
Prepare and submit the final artifact	D − 1 day	Adam Khazem	The document is checked against the published P3 instructions, exported and committed; submission files and repository references are verified.

These are internal planning targets. The published P3 instructions determine the required deliverables and submission deadline.

10.3 Risks

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1 - Reports become outdated before students reach a room.	Open; expiration logic checked in the local spike. Browser behavior and the usefulness of the 60-minute threshold remain unverified.	High / High	Tien: Display reported-status labels and timestamps; implement expiration in open views. Jeimy: Gather feedback on the expiration period.

R2 - Too few students submit reports to provide useful occupancy information.	Open; participation assumptions remain unvalidated.	High / High	Jeimy: Gather student feedback. Keep reporting brief and the catalog usable when reports are Unknown.
R3 - Catalog details are inaccurate, incomplete or unsuitable for publication.	Open; source verification and publication suitability require evidence.	Medium / High	Roshan: Check sample records against identifiable sources and record source dates. Clearly label synthetic demonstration data.
R4 - Additional features or design complexity exceed the course timeline.	Open; scope exclusions and the single-backend design limit complexity.	Medium / High	Adam: Require a documented change decision before adding features; prioritize requirements and preserve agreed exclusions.
R5 - Anonymous users submit false or repeated occupancy reports.	Open; anonymous reporting does not verify truthfulness or prevent repeated valid submissions.	Medium / High	Malak: Maintain reported-status wording and timestamps; verify that invalid room identifiers and observations are rejected.
R6 - Concurrent submissions produce inconsistent acceptance order or incorrect success acknowledgments.	New architectural risk; local SQLite checks passed, but PostgreSQL behavior remains unverified.	Medium / High	Roshan: Verify transaction ordering and rollback on PostgreSQL. Confirm success only after storage commits.
R7 - Late contributions or integration errors leave inconsistent sections or incomplete submission evidence.	Open during final integration.	High / High	Adam: Track owners and deadlines, review cross-section consistency, preserve individual commits and verify contribution links and submission files.

10.4 Repository

Repository: https://github.com/BigKhaz15/RoomRadar

Folder	Contents
/docs/p1/	Previous P1 submission.
/docs/p2/	P2 document and final submission PDF.
/docs/p2/diagrams/	Editable diagram sources and exported images.
/spikes/	Prototype code, reproduction instructions and recorded measurements.

Members commit their authored work on individual branches and open pull requests linked to contribution issues. Adam reviews and merges contributions, then records final integration corrections in separate commits. Individual contribution evidence and assigned review issues are linked in section 0.1. The repository is public and readable without signing in.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Entire Team	ChatGPT / Codex Claude	Assisted with final consistency review, revisions to use cases and analysis models, architecture documentation, traceability and project planning. Generated the replacement domain diagram and prepared and executed the replacement architecture spike.	Compared revisions against the established requirements during integration. Spike code, measurements and execution environment are documented in /spikes/. Results cover a local SQLite prototype; PostgreSQL, browser behavior and hosted performance remain unverified.

P1 AI tool use retained for continuity

Tool	What it was used for	Who used it	How the output was checked
ChatGPT	Assisted with proposal drafting and revision, requirements clarification, diagrams, and project organization.	Noor S, Adam K	Reviewed suggestions against course instructions and project scope; checked consistency, testability, and requirement-to-use-case links, and corrected unsuitable suggestions.
Gemini	Assisted in gathering research	Roshan P, Adam K	Checked by all other team members against the agreed scope.

11 Open Questions

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	Are there restrictions on publishing room information or anonymous reports?	Team	Roshan; D − 10 days	Unverified; applicable rules require confirmation.	Use clearly labeled sample data and make no university-endorsement claim.
Q2	Which rooms have verifiable locations, capacities and amenities?	Team	Roshan; D − 10 days	Actual catalog records and sources remain unverified.	Use a small, clearly labeled sample catalog.
Q3	Will students report occupancy, and is the catalog useful when statuses are Unknown?	Team	Jeimy; D − 10 days	Student participation and perceived usefulness remain unvalidated.	Keep reporting optional and catalog browsing available without reports.

Q4	Is the 60-minute report lifetime useful?	Team	Jeimy and Tien; D − 9 days	The rule is defined, but its usefulness requires student feedback.	Retain 60 minutes until a documented change is approved.
Q5	Which stack, database and host can the team maintain?	Team	Adam and Roshan; D − 9 days	ADR-01 selects HTML/CSS/JavaScript, FastAPI and PostgreSQL. Hosting and cost remain unresolved.	Use the selected stack as the design baseline; do not claim deployment feasibility has been demonstrated.
Q6	Does report acceptance preserve ordering and reject failed writes on the selected database?	Team	Roshan; D − 5 days	Local SQLite checks passed; PostgreSQL behavior remains unverified.	Follow ADR-02's transaction design and acknowledge only committed reports.
Q7	Does open-view expiration remain correct under network delay and browser suspension?	Team	Tien; D − 5 days	Expiration logic passed local boundary checks; browser behavior remains unverified.	Follow ADR-03's server-supplied timing and recheck status when a suspended view resumes.
Q8	Has every P1 feedback item received a documented response?	Team	Adam; before P2 submission	Addressed in section 0.2: each P1 feedback item has a documented response. Outstanding student-feedback, room-data and hosting-cost evidence remains identified in Q1–Q5.	Retain the documented limitations until supporting evidence is collected.

12 Baseline v1.0 Declaration

We declare version 1.0 of this document the P2 baseline for RoomRadar. Its requirements, analysis models and high-level design guide subsequent P3 work. Documented limitations and open questions remain subject to verification. Later baseline changes are recorded in section 0.2 with their reasons and affected IDs.

Team member	Signature or initials	Date
Noor Suhwail	NS	11/08/2026
Adam Khazem	AK	11/08/2026

Raghameena Ramasubbu	RR	11/08/2026
Roshan Palani	RP	11/08/2026
Tien Nguyen	TN	11/08/2026
Jeimy Rodriguez	JR	11/08/2026
Malak Zaarour	MZ	11/08/2026